Agenda :- SQL v/s NoSQL

↳ When not to use SQL.

Key Prop. of SQL DB

1. Normalized & Structured Data

every table has a predefined structure ⇒ cols in each row,
size of cols. / each row,
data type of cols.
indices.
⋮

Normalized ⇒ Create table in a way to nullify redundation as much as possible

c) one data should be stored at only one place.

(2.) ACID Prop. [Transactions].

Atomicity — A → Either complete OR nothing

Consistency — C → Data should be correct.

Isolation — I → I am the only one.

Durability — D → No loss when once saved.

NoSQL → any thing & everything apart from SQL.

↓

SQL-DB

SQL-DB
w/ some of the above prop./contraints relaxed, so as to give additional prop. that SQL doesn't support.

No SQL DB
- fast Writes
- use 1° mem./RAM
- compromise Durability

2° mem v/s 1° mem
↓
80 times slower
↓ ↓
0.01 sec ~ 0.00125 sec
↓
useful
huge vol. writes

Redis

⊛ When to go for NoSQL-DB :-

Will SQL work for me
- YES → go for SQL
- No → Rethink

Dawn Sure

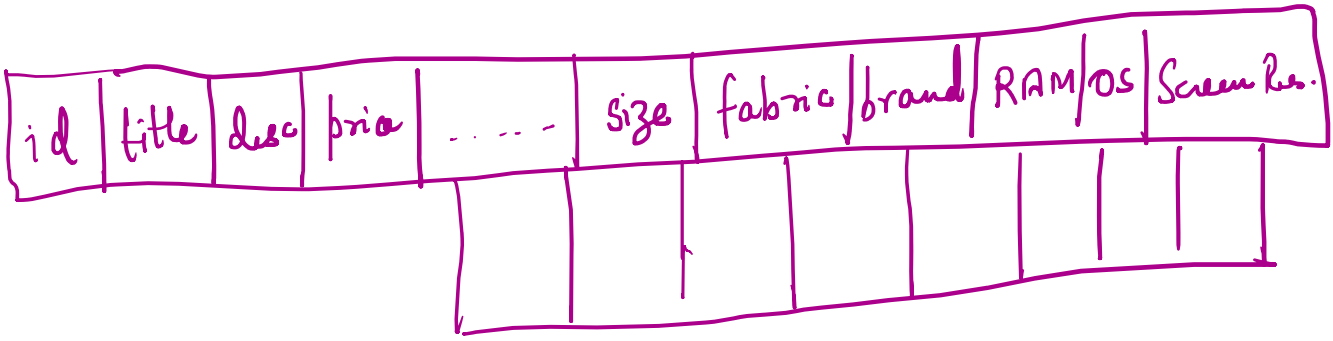


(*) Drawbacks of SQL ?

(1.) Fixed Schema is not good for every use-case.

Myntra / Flipkart / Amazon

products



→ If we store products info in a DB w/ fixed schema, we will end w/ most NULLs in each row.

→ storage wastage
→ sparse tables.

↓

We might need DB w/out fixed st..

↳ Document DB

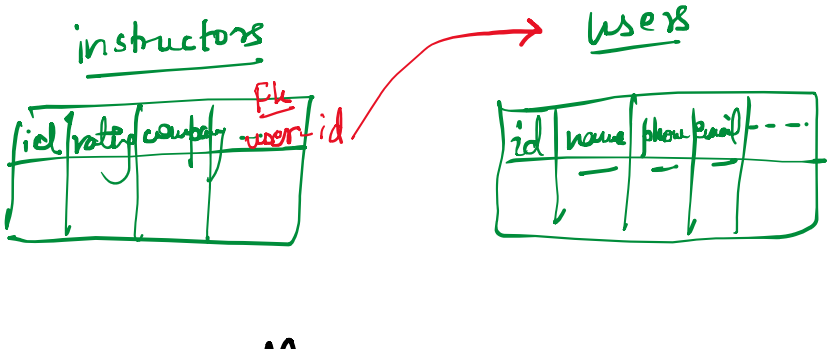
(MongoDB, ElasticSearch)

(2.) We often have to shard data

↳ data is split across multiple m/c's

∴ In SQL, we normalize data

Eg:



↓

Join is inevitable



Joins across machines are impossible.

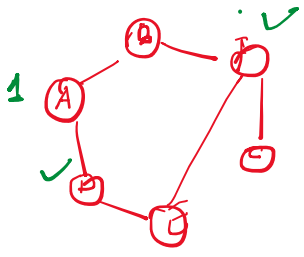


Transactions across shards X

Consider a NoSQL DB.

1:21 pm

⊛ Types of NoSQL.

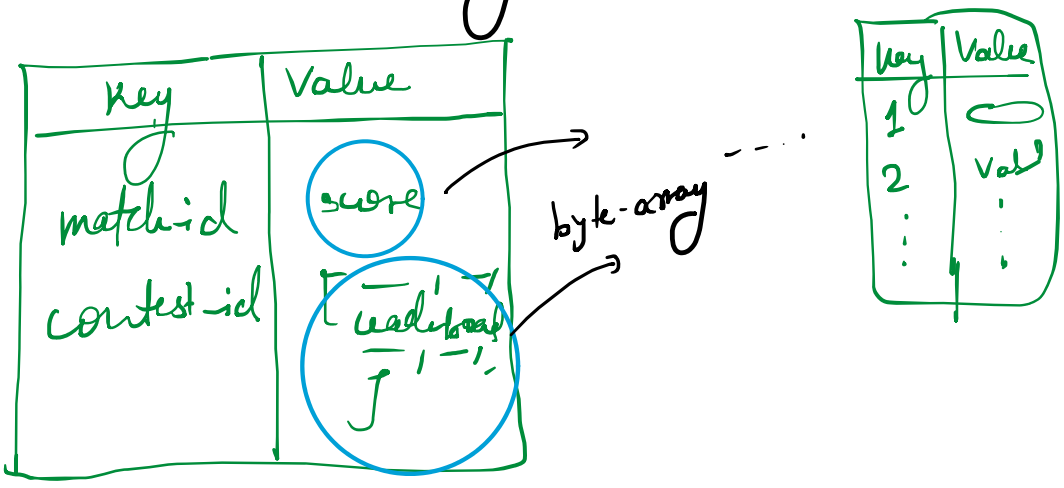


1. Key-value Pairs

↳ Store info like k-v pairs exactly like a hashmap.

↳ Eg. DynamoDB, Redis

When you always want to get the complete value for a particular entity.



- Value has no fixed type
- KV PDB, consider value as a byte[] (byte Array)

→ When you're fetching values as a whole

→ When you're fetching values as a whole
→ When all queries are based on key
=.

2. Document DB.

Myntra eg. covered above
↓
fixed st. lead to → storage wastage
↳ sparse data.

major prob. ⇒ SPARSE TABLE
=.

→ structureless
→ no need to define any st. for any entry.

→ like a k-v pair
BUT but where
you can query on attr. of value

→ Eg. MongoDB, Elastic Search.

each value can have its own set of attr.

attrs.

values are stored like JSON

products : [

101 → { id : 101,
title: —,
desc: —,
OS: —,
color: —,
RAM: —
},

107 → { id : 107,
title: —,
desc: —,
fabric: —,
size: —,
⋮
},

Find products where fabric = '___';

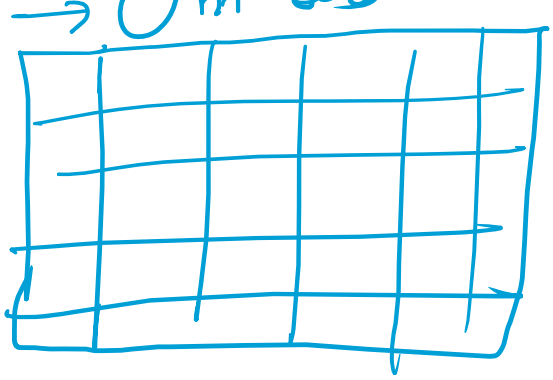
"When each entity may have a diff. set of attributes & you may want to query on attributes."

(3.) Column Family DB

2-D Array

→ m cols

↓ n rows



(I) row-wise

```
for(int i=0; i<n; i++){
    for(int j=0; j<m; j++){
        print(A[i][j]);
    }
}
```

(II) col-wise

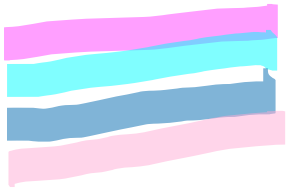
```
for(int i=0; i<m; i++){
    for(int j=0; j<n; j++){
        print(A[j][i]);
    }
}
```

Which is Faster?

(I) row-wise

Time complexity → same

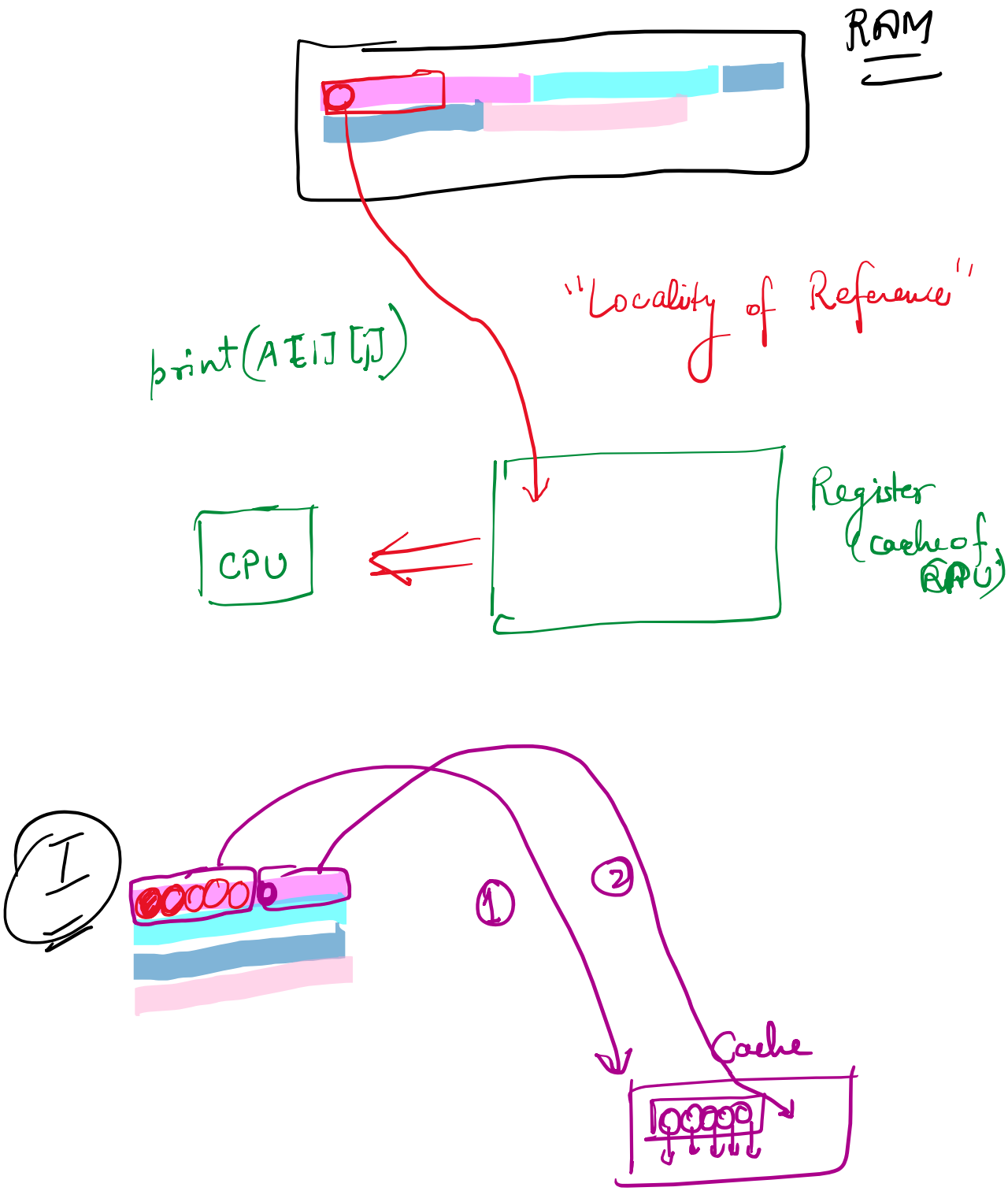
2 Array stored internally



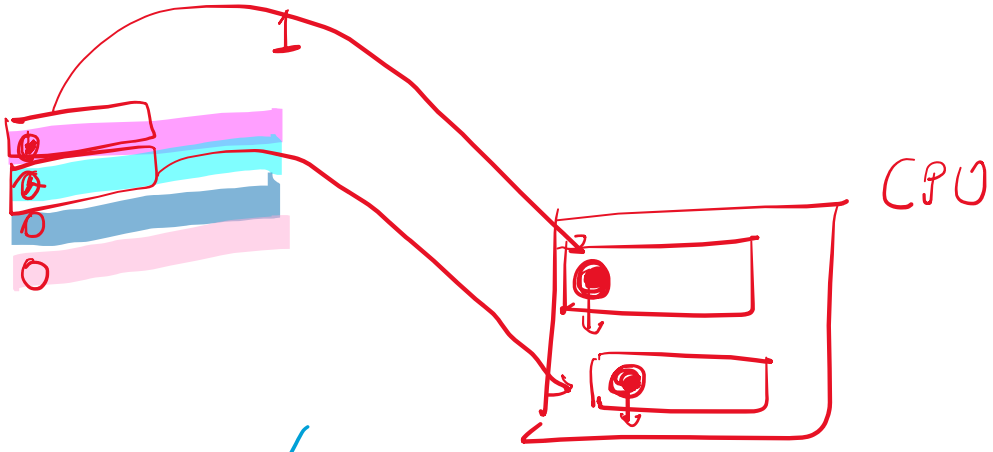


↓ representation

RAM


RAM
"Locality of Reference"
print(A[i][j])
CPU
Register
(cache of RAM)
I
1
2
Cache

11



I have to go to RAM for every subsequent REAN

∴ SLOWER =

Now,

employees

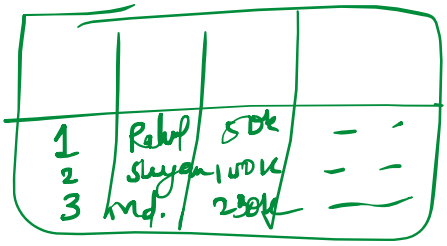
| id | name | salary | age | | |

```
double salary=0; int ct=0;
for (Employee e : employees) {
    salary += e[sal];
    ct++;
}
print (salary/ct);
```

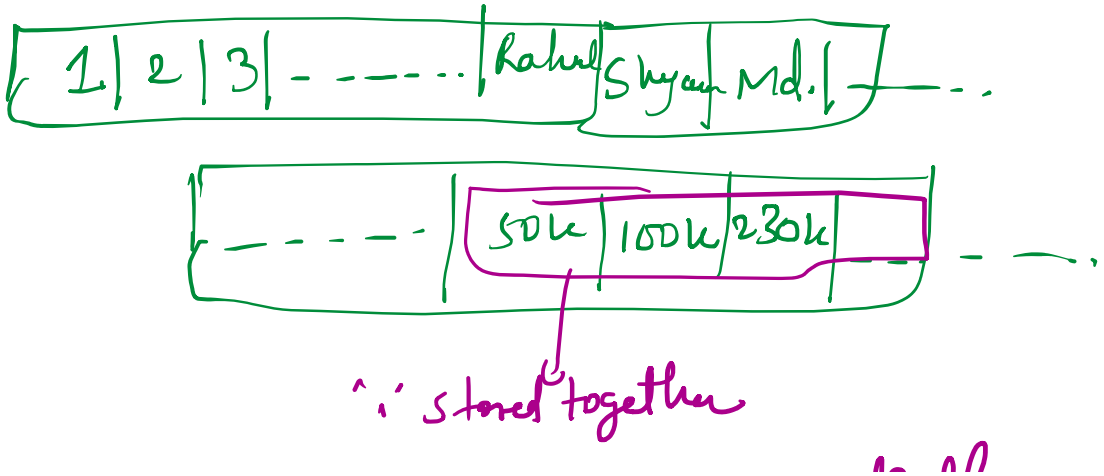
Row-wise of ==

What if data was stored col-wise

Instead of



you store like



∴ stored together

∴ they are essentially

~~row-wise~~
Col-wise

but in single row.

∴ overall you're doing
less disk fetch.

∴ Col^{m} Family DB

↳ Instead of storing row by row,
store data col. by col.

When this is helpful

① Analytical Queries Vol. is ↑

② Mostly I'm interested in
a particular col.

(2) Process a particular col. @ a time

Eg. → Cassandra, HBase, ---etc.

Adv :- Col Family DB

→ Data Compression

↓

lower disk utiliz^{n}

Dataset :-

Shorten string

aaaabbbbxxxxxxxxyyyyllll

O/P → a4b4x7y3l9 ---

136th
36th

A — 150
B — 30

name

Abhishek | Abhishek | Ab - - - -

2033

